

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №7

Тема: Алгоритми пошуку. Загальна класифікація та принципи роботи

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Ознайомитися із загальною класифікацією алгоритмів пошуку даних, їхніми принципами роботи, умовами застосовності та обчислювальною складністю; навчитися обирати найдоцільніший алгоритм пошуку залежно від характеристик даних; дослідити на практиці роботу алгоритмів пошуку стрибками та інтерполяційного пошуку в порівнянні з бінарним пошуком.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Постановка задачі пошуку. Дано набір даних (масив, список, дерево, таблиця) та ключ пошуку. Потрібно визначити, чи міститься елемент із заданим ключем у наборі, і якщо так — знайти його місцезнаходження (індекс, вузол, посилання).

2. Класифікація алгоритмів пошуку за принципом роботи:

- **Послідовний (лінійний) пошук** — перебір усіх елементів підряд до знаходження потрібного або до кінця набору. Не вимагає впорядкованості даних. Складність $O(n)$.
- **Пошук у впорядкованих даних** (вимагає попереднього сортування):
 - **Бінарний (двійковий) пошук** — поділ відрізка пошуку навпіл на кожному кроці. Складність $O(\log n)$.
 - **Інтерполяційний пошук** — оцінка ймовірної позиції елемента на основі лінійної інтерполяції значень (ефективний для рівномірно розподілених числових даних). У середньому $O(\log \log n)$, у гіршому випадку $O(n)$.
 - **Пошук стрибками (Jump search)** – рух блоками фіксованого розміру (зазвичай $\sqrt{n}$) з подальшим лінійним пошуком у знайденому блоці. Складність $O(\sqrt{n})$.
 - **Пошук Фібоначчі** — використовує числа Фібоначчі для поділу відрізка (уникає операції ділення, історично корисно для апаратних обмежень). Складність $O(\log n)$.
- **Пошук у деревоподібних структурах** — бінарне дерево пошуку (BST): $O(\log n)$ у збалансованому дереві, $O(n)$ у виродженому (незбалансованому) випадку; збалансовані дерева (AVL, red-black) гарантують $O(\log n)$.
- **Пошук за хеш-таблицею (хешування)** — обчислення хеш-функції від ключа для прямого доступу до комірки. Середня складність $O(1)$, у гіршому випадку (при колізіях) — $O(n)$.

3. Порівняльна таблиця складності:

Алгоритм	Вимагає сортування	Складність (сер.)	Складність (найгірший)	Додаткова пам'ять
Лінійний пошук	Ні	$O(n)$	$O(n)$	$O(1)$
Бінарний пошук	Так	$O(\log n)$	$O(\log n)$	$O(1)$
Інтерполяційний пошук	Так (рівномірний розподіл)	$O(\log \log n)$	$O(n)$	$O(1)$
Пошук стрибками	Так	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(1)$
Пошук у BST	Так (структурно)	$O(\log n)$	$O(n)$	$O(n)$
Хеш-таблиця	Ні	$O(1)$	$O(n)$	$O(n)$

4. Критерії вибору алгоритму пошуку: розмір набору даних, чи впорядковані дані і чи виправдані витрати на сортування, частота повторних пошуків (окупність попередньої підготовки — сортування, побудови хеш-таблиці чи дерева), розподіл значень ключів (рівномірний чи ні), обмеження на додаткову пам'ять, необхідність підтримки динамічних вставок/видалень.

Хід роботи

Завдання 1. Класифікація алгоритмів пошуку

Заповнити таблицю для семи алгоритмів пошуку: лінійний пошук, бінарний пошук, інтерполяційний пошук, пошук стрибками, пошук Фібоначчі, пошук у бінарному дереві пошуку, пошук у хеш-таблиці. Для кожного вказати: чи вимагає впорядкованих даних; асимптотичну складність у середньому та найгіршому випадках; обсяг додаткової пам'яті; типову область застосування.

Порядок виконання:

1. Створіть таблицю зі стовпцями: "№", "Алгоритм", "Потребує сортування?", "Складність (сер.)", "Складність (найгірший)", "Додаткова пам'ять", "Типове застосування".
2. Для кожного алгоритму зі списку скористайтесь теоретичними відомостями роботи, щоб заповнити стовпці складності та вимог до впорядкованості.
3. У стовпці "Типове застосування" наведіть коротку практичну ситуацію (наприклад: "хеш-таблиця — пошук за ключем у словнику/базі даних з частими вставками").
4. Зверте отриману таблицю з порівняльною таблицею в теоретичній частині та за потреби доповніть її власними прикладами.

Завдання 2. Вибір алгоритму пошуку для заданих сценаріїв

Для кожного з наведених нижче сценаріїв обрати найдоцільніший алгоритм пошуку та обґрунтувати вибір (2–3 речення):

1. Невпорядкований масив із 50 елементів, пошук виконується один раз.
2. Відсортований масив із 10^6 елементів, пошук виконується мільйони разів поспіль.
3. Відсортований масив рівномірно розподілених цілих чисел (наприклад, номери сторінок $1 \dots 10^6$), потрібна максимальна швидкість пошуку.
4. Динамічна структура даних, у яку часто додаються й видаляються елементи, а пошук повинен виконуватись у середньому за $O(1)$.

5. Пристрій із обмеженою обчислювальною потужністю, де операція ділення є "дорогою", а масив відсортований і великий.

Порядок виконання:

1. Для кожного сценарію випишіть ключові характеристики даних (розмір, впорядкованість, розподіл, частота запитів, обмеження).
2. Порівняйте ці характеристики з умовами застосовності алгоритмів із таблиці завдання 1.
3. Оберіть алгоритм, який найкраще відповідає всім вказаним умовам сценарію.
4. Запишіть обґрунтування вибору, посилаючись на конкретну характеристику складності або вимогу (наприклад: "оскільки пошук виконується мільйони разів, варто один раз відсортувати масив і надалі використовувати бінарний пошук — витрати на сортування $O(n \log n)$ окупляться за рахунок швидкості наступних запитів").

Завдання 3. Програмна реалізація пошуку стрибками (Jump Search)

Реалізувати мовою Python алгоритм пошуку стрибками для відсортованого масиву та перевірити його коректність.

Порядок виконання:

1. Оголосіть функцію `jump_search(arr, target)`, яка обчислює розмір блоку `step = int(sqrt(n))`, де `n` — довжина масиву.
2. Заведіть змінні меж поточного блоку `prev = 0` та `curr = step`.
3. У циклі `while curr < n`: перевіряйте, чи `arr[min(curr, n) - 1] >= target`; якщо так — блок знайдено, вийдіть з циклу; інакше зсуньте `prev = curr` і `curr += step`.
4. Після виходу з циклу виконайте лінійний пошук елемента `target` у діапазоні від `prev` до `min(curr, n)`.
5. Якщо елемент знайдено – поверніть його індекс, інакше поверніть `-1`.
6. Підрахуйте кількість порівнянь, виконаних алгоритмом (для подальшого порівняння в завданні 4).
7. Перевірте роботу функції на невеликому тестовому масиві та порівняйте результат з очікуваним індексом (наприклад, за допомогою `arr.index(target)`).

Завдання 4. Програмна реалізація інтерполяційного пошуку та порівняння трьох алгоритмів

Реалізувати інтерполяційний пошук, порівняти середню кількість порівнянь бінарного пошуку, пошуку стрибками та інтерполяційного пошуку на рівномірно розподілених відсортованих даних різного розміру, побудувати графік.

Порядок виконання:

1. Оголосіть функцію `interpolation_search(arr, target)` з межами `low = 0, high = len(arr) - 1`.
2. У циклі, поки `low <= high` і `target` перебуває в діапазоні значень `[arr[low], arr[high]]`, обчисліть ймовірну позицію за формулою `pos = low + (target - arr[low]) * (high - low) // (arr[high] - arr[low])`.
3. Порівняйте `arr[pos]` з `target`: якщо рівні — поверніть `pos`; якщо `arr[pos] < target` — зсуньте `low = pos + 1`; інакше — `high = pos - 1`.
4. Додайте обробку випадку `arr[high] == arr[low]` (щоб уникнути ділення на нуль).

5. Напишіть (або використайте раніше написану) функцію `binary_search(arr, target)`, що також рахує кількість порівнянь.
6. Для кожного розміру масиву $n \in \{10^3, 10^4, 10^5, 10^6\}$ згенеруйте рівномірно розподілений відсортований масив (наприклад, парні числа 0, 2, 4, ...), виберіть 200 випадкових цілових значень і обчисліть середню кількість порівнянь для всіх трьох алгоритмів (бінарний, стрибками, інтерполяційний).
7. Занесіть результати у таблицю "n — бінарний — стрибками — інтерполяційний".
8. Побудуйте графік залежності середньої кількості порівнянь від n (вісь n — у логарифмічному масштабі) для всіх трьох алгоритмів на одному малюнку.
9. Проаналізуйте графік і зробіть висновок про переваги й обмеження кожного з трьох методів.

Приклад реалізації мовою Python:

```
python
import math, random

def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    comparisons = 0
    while low <= high:
        comparisons += 1
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid, comparisons
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comparisons

def jump_search(arr, target):
    n = len(arr)
    step = int(math.sqrt(n))
    comparisons = 0
    prev, curr = 0, step
    while curr < n:
        comparisons += 1
        if arr[min(curr, n) - 1] >= target:
            break
        prev = curr
        curr += step
    for i in range(prev, min(curr, n)):
        comparisons += 1
        if arr[i] == target:
            return i, comparisons
        if arr[i] > target:
            break
    return -1, comparisons

def interpolation_search(arr, target):
    low, high = 0, len(arr) - 1
    comparisons = 0
    while low <= high and arr[low] <= target <= arr[high]:
        comparisons += 1
        if arr[high] == arr[low]:
            return (low, comparisons) if arr[low] == target else (-1, comparisons)
        pos = low + (target - arr[low]) * (high - low) // (arr[high] - arr[low])
        pos = max(low, min(high, pos))
        if arr[pos] == target:
            return pos, comparisons
        elif arr[pos] < target:
```

```

        low = pos + 1
    else:
        high = pos - 1
    return -1, comparisons

# Порівняння середньої кількості порівнянь
for n in [10**3, 10**4, 10**5, 10**6]:
    arr = list(range(0, n * 2, 2))
    targets = random.sample(arr, 200)
    avg = {"binary": 0, "jump": 0, "interpolation": 0}
    for t in targets:
        avg["binary"] += binary_search(arr, t)[1]
        avg["jump"] += jump_search(arr, t)[1]
        avg["interpolation"] += interpolation_search(arr, t)[1]
    for k in avg:
        avg[k] /= len(targets)
    print(f"n={n:>8}: binary={avg['binary']:.2f} jump={avg['jump']:.2f} interpolation={avg['interpolation']:.2f}")

```

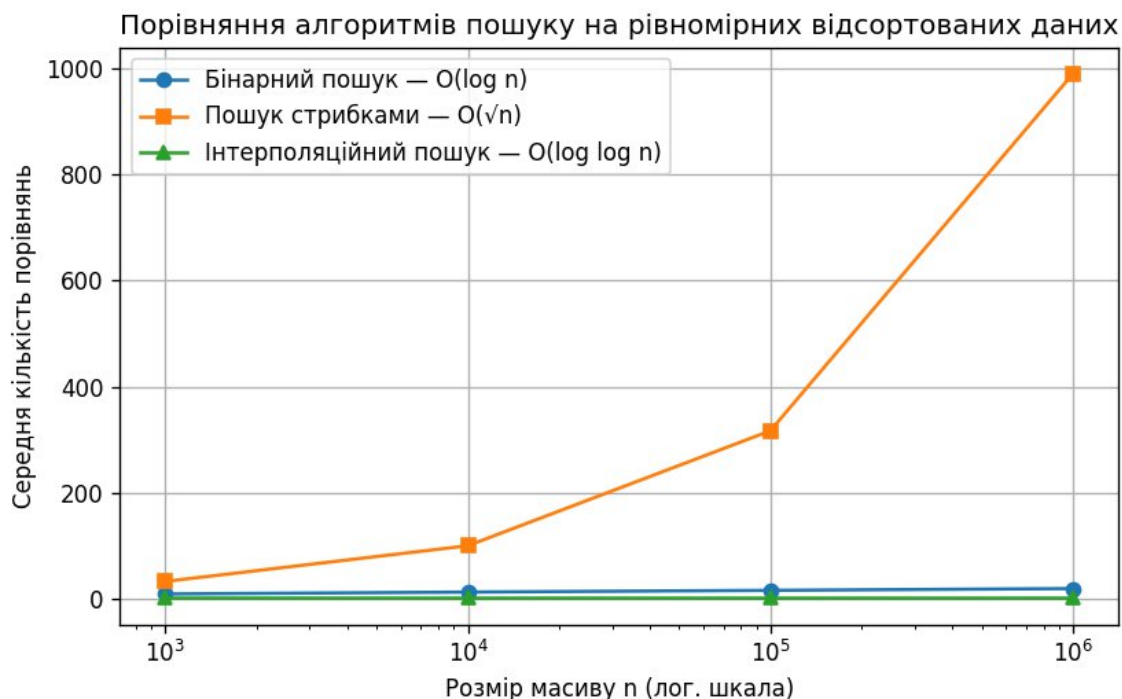
Результат виконання програми:

```

n= 1000: binary=8.81 jump=32.42 interpolation=1.00
n= 10000: binary=12.38 jump=99.79 interpolation=1.00
n= 100000: binary=15.62 jump=316.80 interpolation=1.00
n= 1000000: binary=18.97 jump=990.54 interpolation=1.00

```

Графік порівняння:



Висновок за результатами: на рівномірно розподілених даних інтерполяційний пошук знаходить елемент практично за 1 порівняння незалежно від розміру масиву (теоретично $O(\log \log n)$) – формула інтерполяції одразу "влучає" в потрібну позицію. Бінарний пошук зростає повільно й передбачувано за законом $\log n$. Пошук стрибками, хоча й ефективніший за лінійний, суттєво програє обом іншим методам при зростанні n (закон $\sqrt{n}$ зростає значно швидше за $\log n$), що підтверджує теоретичну класифікацію складності з

таблиці завдання 1. Водночас варто пам'ятати, що перевага інтерполяційного пошуку проявляється лише на рівномірно розподілених даних – на нерівномірних даних його ефективність різко падає до $O(n)$.

Контрольні питання

1. Сформулюйте загальну постановку задачі пошуку.
2. За якими критеріями класифікують алгоритми пошуку?
3. У чому принципова відмінність лінійного пошуку від бінарного?
4. Поясніть принцип роботи інтерполяційного пошуку. За яких умов він найефективніший?
5. Опишіть принцип роботи алгоритму пошуку стрибками. Чому оптимальний розмір блоку дорівнює $\sqrt{n}$?
6. Яка складність пошуку в незбалансованому та збалансованому бінарному дереві пошуку?
7. Чому середня складність пошуку в хеш-таблиці $O(1)$, а найгірша — $O(n)$?
8. Якими критеріями слід керуватися при виборі алгоритму пошуку для конкретної задачі?

Зміст звіту

1. Тема, мета роботи.
2. Заповнена таблиця класифікації (завдання 1).
3. Обґрунтування вибору алгоритмів для сценаріїв (завдання 2).
4. Тексти програм та результати їх роботи (завдання 3, 4).
5. Графік порівняння алгоритмів пошуку.
6. Висновки.